

Ejercicio 1

El algoritmo de *backpropagation* cumple con las siguientes opciones:

- Es un método de regularización para redes neuronales.
- Es un algoritmo para el cálculo del gradiente de una función.

Ejercicio 2

Inciso a)

PyTorch permite diferenciar funciones por medio de **diferenciación automática**, con el método *torch.autograd*.

Inciso b)

PyTorch permite:

1. Paralelizar operaciones en múltiples procesadores de CPU y GPU.
Se llama paralelismo de datos. Distribuye el trabajo computacional.

Ejercicio 3

Sea la función

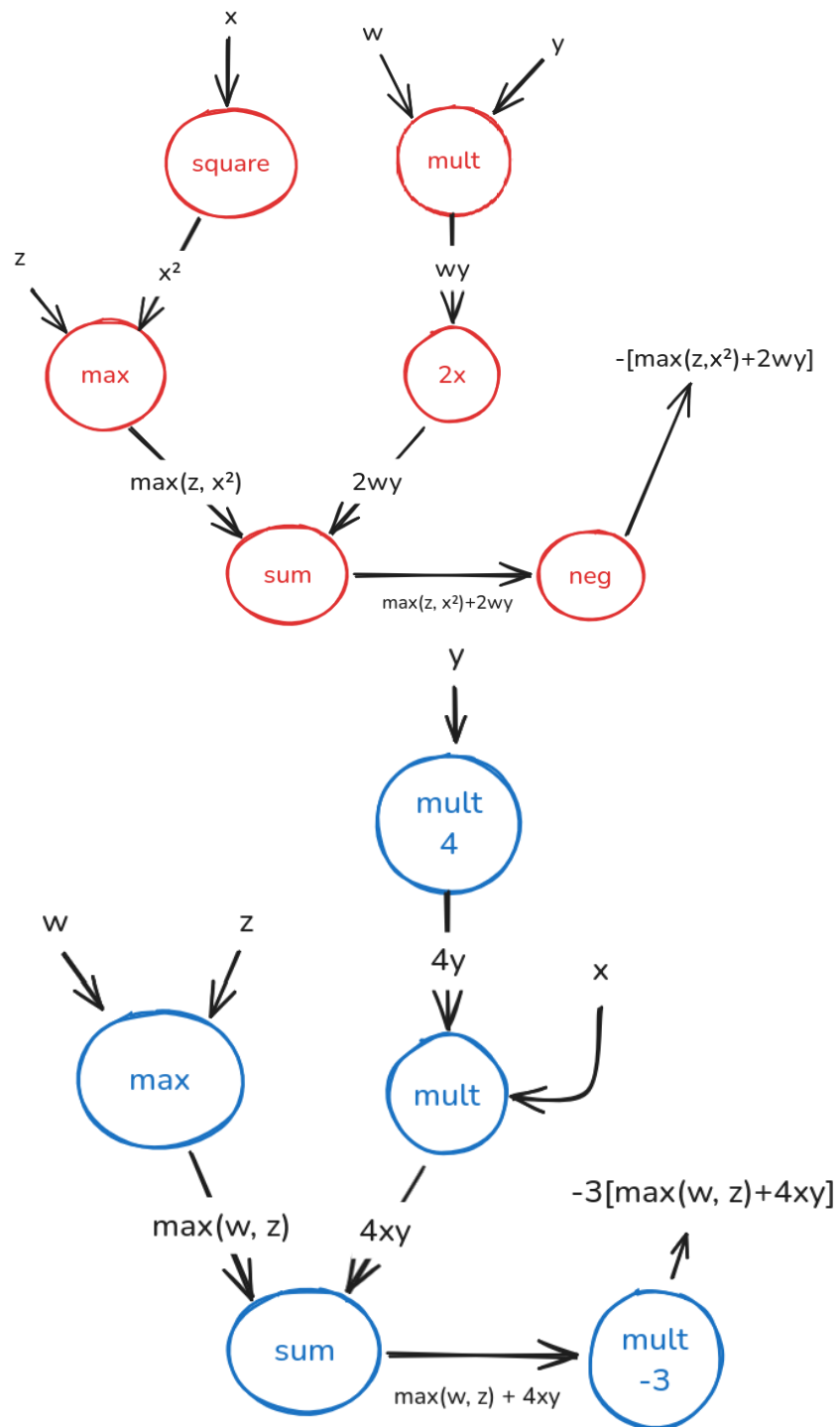
$$f(w, x, y, z) = -(\text{máx}(z, 2x) + 2(wy)) \quad (1)$$

evaluada en el punto $w = 3, x = 2, y = 4, z = 1$, y la función

$$g(w, x, y, z) = -3((4xy) + \text{máx}(w, z)) \quad (2)$$

evaluada en el punto $w = 3, x = 5, y = 2, z = 4$.

Inciso 1



Inciso 2**Inciso 3****Ejercicio 4****Inciso 1**

$$\delta_j^L = (a_j^L - y_j^L) \sigma'(z_j^L)$$

Demostración:

$$C_j = \frac{1}{2}(y_j^L - a_j^L)^2$$

$$\frac{\partial C}{\partial a_j^L} = \frac{\partial}{\partial a_j^L} \left[\frac{1}{2}(y_j^L - a_j^L)^2 \right] = (a_j^L - y_j^L)$$

Como $a_j^L = \sigma(z_j^L)$, entonces

$$\frac{\partial a_j^L}{\partial z_j^L} = \sigma'(z_j^L)$$

Aplicando la regla de la cadena:

$$\delta_j^L = \frac{\partial C}{\partial z_j^L} = \frac{\partial C}{\partial a_j^L} \cdot \frac{\partial a_j^L}{\partial z_j^L} = (a_j^L - y_j^L) \sigma'(z_j^L)$$

Inciso 2

$$\delta_j^l = \sigma'(z_j^l) \sum_k \delta_k^{l+1} w_{jk}^{l+1}$$

Usamos la igualdad de la guía:

$$\delta_j^l = \sum_k \frac{\partial C}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l}$$

Por definición:

$$z_k^{l+1} = \sum_j w_{jk}^{l+1} a_j^l + b_k^{l+1} = \sum_j w_{jk}^{l+1} \sigma(z_j^l) + b_k^{l+1}$$

Por lo tanto:

$$\frac{\partial z_k^{l+1}}{\partial z_j^l} = w_{jk}^{l+1} \sigma'(z_j^l)$$

Entonces:

$$\delta_j^l = \sum_k \delta_k^{l+1} \cdot w_{jk}^{l+1} \sigma'(z_j^l)$$

Factorizando:

$$\delta_j^l = \sigma'(z_j^l) \sum_k \delta_k^{l+1} w_{jk}^{l+1}$$

Inciso 3

$$\frac{\partial C}{\partial w_{ij}^l} = \delta_j^l a_i^{l-1}$$

Demostración: Como

$$z_j^l = \sum_i w_{ij}^l a_i^{l-1} + b_j^l,$$

entonces:

$$\frac{\partial z_j^l}{\partial w_{ij}^l} = a_i^{l-1}$$

Usando la regla de la cadena:

$$\frac{\partial C}{\partial w_{ij}^l} = \frac{\partial C}{\partial z_j^l} \cdot \frac{\partial z_j^l}{\partial w_{ij}^l} = \delta_j^l a_i^{l-1}$$

Inciso 4

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l$$

Como:

$$z_j^l = \sum_i w_{ij}^l a_i^{l-1} + b_j^l,$$

entonces:

$$\frac{\partial z_j^l}{\partial b_j^l} = 1$$

Por lo tanto:

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \cdot 1 = \delta_j^l$$

Interpretación de los δ

Los valores δ_j^l representan el *error* que la neurona j de la capa l contribuye al costo total. Cada δ mide cuán sensible es el costo ante un cambio en z_j^l , permitiendo así propagar el error hacia atrás durante el algoritmo de backpropagation.